BIOINFORMATICS

Workshop 3: A Markov Chain and HMM Class, and a CpG-Island Scanner

Prof. Magnus Bordewich

Learning Objectives

- Estimate a Markov chain's transition matrix from labelled training sequences and build a CpG-island log-odds scanner.
- Implement a small, reusable HMM class in Python.
- Simulate (state, symbol) sequences from an HMM and compute the joint probability of a state path and emission sequence.

Background

A Markov chain is defined by an initial distribution and a transition matrix M; ML estimation from labelled data is simple counting. An HMM extends this with hidden states and emission probabilities E. This workshop builds the HMM class that Workshops 4-5 will reuse for decoding, evaluation and parameter learning.

Coding Activity

1. Write log_odds_score(seq, M_plus, M_minus) matching the lecture's RATIO(x) formula, and a sliding-window scan(seq, M_plus, M_minus, window) that reports it across a long test sequence. Test on the supplied data sets to find CpG islands.
2. Write a class HMM(states, init, trans, emit) storing an initial-distribution dict and nested transition/emission dicts trans[k][l], emit[k][b].
3. Add a method .simulate(length) that samples a (state_path, emission_sequence) pair following the generative process from the lecture, and a method .joint_prob(states, symbols) computing p(s,x) = product_i m[s(i-1)][si] * e[si][xi].
4. Build an HMM for the occasionally dishonest casino. Generate sequences with heavily loaded and slightly loaded dice and see if you can visually obsreve the effect.
5. Build an eight-state CpG-island HMM with illustrative parameters making islands short and CG-rich, simulate a length-200 sequence, and print both the hidden path and the emitted letters.

Extension Tasks (Optional)

a) Write estimate_transition_matrix(seqs) that, given a list of DNA sequences, counts every consecutive letter pair and returns the maximum-likelihood transition matrix M (a dict of dicts).

b) Investigate how window size affects your CpG scanner's ability to localise island boundaries.

Looking Ahead

Keep your HMM class — next lecture's forward, backward and Viterbi algorithms are naturally implemented as methods on it.